

```
print(t.linearize())
(-32,(2),(1,(23,(4),(-2)),(12)))
True
False
(0,(0),(0,(0,(0),(0)),(0)))
(1024,(4),(1,(529,(16),(4)),(144)))
[2, -32, 4, 23, -2, 1, 12]
```

Notera att alla funktioner som vi definierat är rekursiva. Detta exempel visar hur vi på ett enkelt sätt kan kombinera klasser och rekursion för att representera och modifiera strukturerad data.

11.4 Uppgifter

1. Vilka av exemplen i kapitel 10 är exempel på enkelt, mångnivå, hierarkiskt och hybrid arv?
2. Flytta `__str__` metoden för `Plus` och `Times` till `BinOp` på lämpligt sätt så att vi inte har någon kodduplikation. Exempen ska fortfarande skrivas ut på rätt sätt.
3. Lägg till en klass `Exp` för exponentiering av algebraiska uttryck. Den ska binda hårdare än både `+` och `*`, samt skrivas ut som operatoren `**` (dvs på samma sätt som i Python).
4. Lägg till en funktion `big_constant_sum(n,e)` där `n` är ett positivt heltal och `e` ett algebraiskt uttryck. Funktionen ska skapa ett uttryck där `e` adderas med sig själv `n` gånger. Om `n` är `0` så ska konstanten `0` returneras. Så funktionen ska alltså returnera summan

$$\sum_{i=0}^{n-1} e$$

5. Lägg till en funktion `big_sum(n,e)` där `n` är ett positivt heltal och `e` en funktion från heltal till algebraiska uttryck. Funktionen ska skapa ett uttryck som representerar

$$\sum_{i=0}^n e_i$$

6. Definera evaluering av algebraiska uttryck givet en uppslagstabell `d` från variabler till siffror. För att göra detta ska ni lägga till en metod `evaluate(d)` till `Constant`, `Var`, `Plus`, `Times` och `Exp`. Ni kan anta att alla variabler i uttrycket finns definierade i `d`.
7. † Implementera rosträd. Dessa är en typ av träd där varje nod innehåller ett värde och en lista med fler rosträd. Ett löv med värdet `5` representeras genom `RoseTree(5, [])`. Implementera en klass för rosträd som har samma metoder som `BinTree`.

Kapitel 12

Defensiv programmering

Den som har lärt sig programmera har nog också blivit tvungen att lära sig felsöka. Alla programmerare gör fel, råkar förlita sig på olämpliga antaganden, och missförstår dokumentation. Fel kan göra att programmet avbryter precis där misstaget är, men det kan också uppstå långt senare. Om division med noll uppstår så kanske det är ett symptom på ett tidigare misstag. En variabel kanske har blivit satt till 0, trots att det är orimligt, men var gjordes det och varför? Det kan vara arbetsamt att leta upp orsaken till felet. Programmeringsstilen *defensiv programmering* syftar dels till att hitta fel tidigt, så nära det egentliga misstaget, för att undvika arbetsamma felsökningssessioner, och dels till att tydliggöra sina antaganden i koden.

Defensiv programmering avser att man förväntar sig att funktioner kommer att användas på fel sätt, algoritmer blir implementerade fel, kombinationer av variablers värden uppstår som man inte har tänkt på, och att andra konstigheter sker under programutveckling. Genom att *koda* sina förväntningar på variablers värden med hjälp av **assert** på många platser i ett program kan man upptäcka felen tidigare, närmare felkällan, och därmed förenkla felsökningen.

12.1 Assert

Instruktionen **assert** condition anger att uttrycket condition förutsattes vara sant. Om uttrycket är sant händer inget, det är det förväntade uppträdandet, men om det är falskt avbryter programmet med hjälp av ett särfall, **AssertionError**. Denna typ av instruktion finns i många språk och är ett hjälpmedel för felsökning, utveckling, och underhåll av program.

Några exempel:

```
assert len(data_list) > 0
assert x != 0
assert x > 0 and y > 0
```

Dessa rader ställer tre tänkbara krav på tillståndet i ett program. Instruktionen **assert** ska utläsas som "detta gäller".

12.1.1 Avbrott och felmeddelande

Säg att vi har skrivit en divisionsfunktion på följande sätt:

```
def divide(x,y):
    assert y != 0
    return x / y
```

Om vi nu anropar divide med 0 som andra argument

```
print(divide(10,2))
print(divide(10,0))
```

så kommer ett felmeddelande skrivas ut:

```
Traceback (most recent call last):
  File "f12.py", line 6, in <module>
    print(divide(10,0))
  File "f12.py", line 2, in divide
    assert y != 0
AssertionError
```

Notera att särffallet `AssertionError` inte är tänkt att fångas med `except`; man använder `assert` för att hitta allvarliga programmeringsfel, inte för att hantera körfel (som t.ex., namngiven fil finns ej, minnet räcker inte, man har rekurerat för djupt, o.s.v.).

12.1.2 Generellt

Det generella fallet kan skrivas

```
assert condition [, comment_string]
```

där condition är ett boolskt uttryck och comment_string är en möjlig förklarande text. Man kan alltså skriva villkoren ovan som:

```
assert len(data_list) > 0, 'User submitted empty data'
assert x != 0, 'x==0 must be avoided early in the application!'
assert x > 0 and y > 0, 'Both x and y have to be positive!'
```

för att hjälpa till att förstå felen som uppstår.

Exempel:

```
def divide(x,y):
    assert y != 0, "Second argument cannot be 0 when dividing"
    return x / y
print(divide(10,0))
```

ger

```
Traceback (most recent call last):
  File "f12.py", line 6, in <module>
    print(divide(10,0))
  File "f12.py", line 2, in divide
    assert y != 0, "Second argument cannot be 0 when dividing"
AssertionError: Second argument cannot be 0 when dividing
```

12.1.3 Tumregler för assert

assert är implementerat med hjälp av sårfall, men syftet är att uttrycka antaganden i koden, inte att hantera oväntade tillstånd.

- **assert** är för problem under kodning/utveckling.
- **raise** och **try/except** är för oväntade effekter under körning.

Man kan säga att **assert** används för att fånga oväntade tillstånd i koden, **raise** används för att signalera eventuella förväntade fel och **try/except** används för att fånga mer eller mindre förväntade fel. Ett program ska inte vara beroende av **assert**, utan fungera minst lika bra utan. I allmänhet går ett program långsammare med **assert** eftersom det är mer kod att exekvera, men man ska inte dra sig för att lägga till **assert** i sin kod. Om snabbhet är viktigt kan man starta sitt program med `python -O kod.py` istället för `python kod.py`; flaggan `-O` deaktiverar alla **assert**-instruktioner.

12.1.3.1 Exempel 1: remove_trailing_spaces

Nedan följer ett exempel på **assert** i `remove_trailing_spaces`:

```
def remove_trailing_spaces(s, spaces=" \t"):
    assert len(spaces) > 0
    i = len(s)
    while i > 0 and s[i-1] in spaces:
        i -= 1
    return s[0:i]
```

12.1.3.2 Exempel 2: newton_raphson

Nedan följer ett exempel på **assert** i en funktion för att beräkna roten av en funktion med hjälp av Newton-Raphsons algoritm:

```
def newton_raphson(f, f_prime, x, precision):
    assert isinstance(x, float)
    assert isinstance(f(x), float)
    assert isinstance(f_prime(x), float)
    assert 0.0 < precision < 1.0, 'precision must be in the interval (0,1)'

    ready = False
    while not ready:
        x_next = x - f(x)/f_prime(x)      # Main line of code to be repeated
        ready = abs(x-x_next) < precision # Test for stopping criterion
        x = x_next                        # Preparing the next iteration

    return x_next
```

Som du ser kan vi testa typen för ett objekt med hjälp av **isinstance**. Funktionen **type**, som returnerar objektets typ, är också användbar, men **isinstance** är mer generell; **isinstance** kan som andra argument ta en tupel av klasser och returnerar om det första argumentet är en instans av *någon* av klasserna *eller* dess subklasser.

Observera även att **assert** kan ses som extra dokumentation kring de antagande som görs i programmet.

12.1.4 Villkor och invarianter

Vi kan dela upp **assert**-användandet i tre sorter:

- **förvillkor**: uttryck som ska vara sanna när en funktionskropp exekveras (eng. *preconditions*)
- **eftervillkor**: uttryck som ska vara sanna när en funktion returnerar (eng. *postconditions*)
- **loop invarianter**: uttryck som ska vara sanna i en loop (eng. *loop invariants*)

12.1.4.1 Exempel 3

Låt oss säga att vi har en databas som kopplar unika identifierare (representerade som `int`) med namn (representerade som `str`). För att det ska gå snabbt att slå upp någon genom antingen dess identifierare eller namn har vi två uppslagstabeller, en från identifierare till namn och en från namn till identifierare. För att detta ska fungera måste ju nycklarna i den ena vara samma som värdena i den andra, och vice versa.

```
class MyDB:

    def __init__(self):
        self._id2name_map = {}
        self._name2id_map = {}

    def add(self, id, name):
        self._name2id_map[name] = id
        self._id2name_map[id] = name

    def by_name(self, name):
        id = self._name2id_map[name]
        return id
```

En första sak som kan bli fel här är ju att användaren helt enkelt skriver över en av `_id2name_map` och `_name2id_map`, eller att någon annan metod vi senare lägger till har en bugg som bryter mot invarianten för klassen. Vi kan lägga in en **assert** i `by_name` där vi dubbelkollar att invarianten ej har blivit bruten:

```
class MyDB:

    def __init__(self):
        self._id2name_map = {}
        self._name2id_map = {}

    def add(self, id, name):
        self._name2id_map[name] = id
        self._id2name_map[id] = name

    def by_name(self, name):
        id = self._name2id_map[name]
        assert self._id2name_map[id] == name
        return id
```

Om vi nu lägger in en buggig metod eller en användare använder klassen på fel sätt så kommer `by_name` generera ett **AssertionError**. En annan invariant är ju att ett `id` ska vara en `int` och namn `str`. I många programmeringsspråk med striktare typsystem kan vi specificera detta i signaturen för klassen eller

konstruktorn. I Python kan vi lösa detta genom att lägga in en **assert** tillsammans med `isinstance`:

```
class MyDB:

    def __init__(self):
        self._id2name_map = {}
        self._name2id_map = {}

    def add(self, id, name):
        assert isinstance(id, int), ("id is not an integer: " + str(id))
        assert isinstance(name, str), ("name is not a string: " + str(name))
        self._name2id_map[name] = id
        self._id2name_map[id] = name

    def by_name(self, name):
        id = self._name2id_map[name]
        assert self._id2name_map[id] == name
        return id

db = MyDB()
db.add(2,2)
db.add("hej", "du")
```

12.2 Testning

Det är svårt att garantera att ens kod är korrekt. Man kan resonera matematiskt om kod och algoritmer, men även då är det lätt att det blir fel. Oavsett hur mycket man planerar, tänker och granskar, så är det bra att också noggrant testa sin kod. Men hur gör man det på bästa sätt?

Erfarenheten visar att det varken är tillräckligt eller praktiskt att testa om ett helt program, själva *produkten*, fungerar som det ska. Om man vill upptäcka problem tidigt och skydda sig mot fel på grund av ändringar i koden, då bör man testa programmets funktioner och metoder.

Ett vanligt arbetssätt är att (1) skriva lite kod, kanske en funktion, (2) verifiera att koden fungerar, och sen fortsätta skriva mer kod. Det kan vara enkelt att verifiera koden: man väljer några parametrar till sin funktion och ser om resultatet blir som det var tänkt. Ett scenario som i praktiken är ännu vanligare är följande:

1. Ändra i befintlig kod, för att lägga till ny funktionalitet.
2. Verifiera den nya funktionaliteten.
3. Verifiera den gamla funktionaliteten.

Punkt 3 blir lätt bortglömd, men den kan också vara lite jobbig att ta hand om. Det är ju på den nya funktionaliteten man har sitt fokus, inte den gamla, och hur var det nu vi testade den koden? Om du ändrar i en funktion `fun`, kommer du då också ihåg att testa andra delar av din kod som är beroende av `fun`? Allteftersom tiden går och ett projekt växer blir det allt svårare verifiera gammal funktionalitet. Och om man jobbar med andras kod blir det förstås ännu svårare att komma på hur man bör testa de funktioner som finns. Så hur gör man?

Det som behövs är *systematisk* och *automatisk* testning, vilket man åstadkommer genom att koda

testningen, och metodiken för detta kallas *enhetstestning* (eng. *unit testing*). Idag finns det så pass bra stöd för testning i vanliga programmeringsspråk att det ofta är enkelt att inkludera tester i sitt projekt. I det här avsnittet tittar vi på några olika sätt att arbeta med tester.

Ni kan läsa mer på: <https://realpython.com/python-testing/>

12.2.1 Testning med assert

Kodtestning kan göras både manuellt och automatiskt. För enkel manuell testning kan vi använda **assert**.

Låt oss betrakta en liten Python-modul (nedan) för att jobba med polynom. I modulen nedan representeras polynom som en lista, där listan håller värdet på koefficienten till potensen, där index i i listan motsvarar x^i . Till exempel:

Polynom	Python-representation
x^4	[0, 0, 0, 0, 1]
$4x^2 + 5x^3$	[0, 0, 4, 5]
$5 + 4x + 3x^2 + 2x^3 + x^4$	[5, 4, 3, 2, 1]

Modulen nedan innehåller tre funktioner för att utföra beräkningar på polynom. Vi kan beräkna värdet av ett polynom för ett visst värde på x , samt addera och subtrahera polynomen.

```
# My module for handling polynomials

def eval_poly(p_list, x):
    """Evaluates a polynomial at a given value x.
    """
    degree = 0
    val = 0
    for coeff in p_list:
        val += coeff * x**degree
        degree += 1
    return val

def add_poly(p0, p1):
    """Adds polynomials
    """
    poly_add = []
    p0_cpy = p0.copy()
    p1_cpy = p1.copy()
    while p0_cpy and p1_cpy:
        poly_add.append(p0_cpy.pop(0) + p1_cpy.pop(0))
    if p0_cpy:
        poly_add += p0_cpy
    elif p1_cpy:
        poly_add += p1_cpy
    return poly_add
```

```
def sub_poly(p0, p1):
    """Subtracts polynomials
    """
    poly_sub = []
    p0_cpy = p0.copy()
    p1_cpy = p1.copy()
    while p0_cpy and p1_cpy:
        poly_sub.append(p0_cpy.pop(0) - p1_cpy.pop(0))
    if p0_cpy:
        poly_sub += p0_cpy
    elif p1_cpy:
        poly_sub += [-x for x in p1_cpy]
    return poly_sub
```

Med vår polynom-modul kan vi till exempel utföra följande beräkningar:

```
p = [2,0,1]
q = [-2,1,0,0,1]
y1 = add_poly(p, q)
y2 = sub_poly(p, q)
y3 = eval_poly(p, 2)
print(y1, y2, y3, sep = '\n')
```

```
[0, 1, 1, 0, 1]
[4, 0, 0, 0, 1]
6
```

I framtiden kanske vi vill lägga till funktioner för att derivera eller multiplicera polynom, eller beräkna värdet av derivatan i en viss punkt med mera. Vissa av dessa kommande funktioner kan komma att använda sig av (d.v.s. bero på) de tidigare funktionerna vi implementerat. Det är även vanligt att vi inser att vissa funktioner kan behöva ändras, generaliseras eller förenklas för att göra modulen enkel att underhålla.

Till exempel, i modulen ovan verkar `add_poly` och `sub_poly` rätt krångliga. Vi noterar att subtraktion av ett polynom q från ett polynom p är detsamma som att addera negationen $-q$ till p , dvs $p - q = p + (-q)$. Vi inser att implementeringen av negering är enkel, och kan därför förenkla (nästan helt ta bort) koden i `sub_poly`. Inte bara har vi tagit bort en komplex funktion, utan vi har utökat vår modul till att kunna negera polynom. Den nya funktionen `neg_poly` och den omskrivna funktionen `sub_poly` ser vi nedan.

```
def neg_poly(p_list):
    """Negates a given polynomial.
    """
    p_neg = []
    for coeff in p_list:
        p_neg.append(-coeff)
    return p_neg

def sub_poly(p0, p1):
    """Subtracts polynomials
    """
    return add_poly(p0, neg_poly(p1))
```

I detta exempel är poängen att funktioner och klasser i en modul kan komma att ändras och utökas. Även om vi i detta lilla exempel har full koll på de fyra funktionerna i vår polynom-modul så kan stora moduler med hundratals funktioner och klasser där vissa interagerar (t.ex. `sub_poly` och `add_poly` ovan) snabbt kan bli oöverskådliga. Det är därför bra att redan från början tänka på att skapa tester för funktionerna för att se att de bibehåller förväntad funktionalitet. Till exempel kan vi skapa en tillhörande test-fil som vi döper till `test_polynomials.py`, där vi skriver följande kod.

```
import polynomials as pol # Import our polynomials.py module

print("Run tests for polynomial module")
p = [2,0,1]
q = [-2,1,0,0,1]
r = [] # always good to check corner cases

assert (pol.eval_poly(p,0) == 2)
assert (pol.eval_poly(p,1) == 3)
assert (pol.eval_poly(q,2) == 16)
assert (pol.eval_poly(q,-2) == 12)
assert (pol.eval_poly(r,1) == 0)
assert (pol.add_poly(p,r) == p)
assert (pol.add_poly(p,q) == pol.add_poly(q,p))
assert (pol.neg_poly(pol.neg_poly(q)) == q)
assert (all(x == 0 for x in pol.sub_poly(p,p)))
assert (pol.sub_poly(p,q) == [4, -1, 1, 0, -1])
assert (pol.eval_poly(add_poly(p,q),12) == pol.eval_poly(p,12) + pol.eval_poly(q,12))
print("Tests completed successfully.")
```

Kör vi vårt program `test_polynomials.py` kommer tester att exekveras upp till första `assert` som inte går igenom. Om vi modifierar våra funktioner hoppas vi på att någon av testerna fångar upp ifall vi råkat introducera någon bugg. Det finns ingen garanti att vi kommer fånga upp alla möjliga buggar med våra tester, men med väl designade tester kan vi hoppas fånga de flesta.

Ett problem är dock att vi endast ser resultatet av det första `assert` som inte håller. Ett bättre sätt för testning av mer komplexa program är *enhetstester*.

12.2.2 Enhetstester med unittest

Modulen `unittest` har funnits med i Pythons standardbibliotek sedan version 2.1 och används av många projekt, både kommersiella och open-source. För att använda `unittest` måste vi:

1. Gruppera alla tester i klasser där testerna är metoder.
2. Använda specifika `assert`-metoder från `unittest.TestCase` istället för de inbyggda `assert` uttrycken.

För att skriva om exemplet `test_polynomials.py` ovan så att det använder `unittest` gör vi följande:

- Importera `unittest`
- Skapa en klass `TestPoly` som ärver från `TestCase` klassen
- Skriv om alla `assert` till metoder i den här klassen som använder `self.assertEqual()`
- Ändra så att `unittest.main()` körs när filen laddas

Så här kan filen `test_polynomials.py` se ut:

```
import unittest
import polynomials as pol

p = [2,0,1]
q = [-2,1,0,0,1]
r = [] # always good to check corner cases

class TestPoly(unittest.TestCase):

    def test_eval_poly_p0(self):
        self.assertEqual(pol.eval_poly(p,0), 2, "Should be 2")

    def test_eval_poly_p1(self):
        self.assertEqual(pol.eval_poly(p,1), 3, "Should be 3")

    # ...

    def test_add_poly_p_r(self):
        self.assertEqual(pol.add_poly(p,r), p, "Adding empty list should not modify p")

    def test_add_poly_symmetric(self):
        self.assertEqual(pol.add_poly(p,q), pol.add_poly(q,p), "Function is not symmetric")

    def test_eq_poly_q0(self):
        self.assertEqual(pol.sub_poly(p,p), [0 for x in range(len(p))], "Not all 0")

    # ...

if __name__ == '__main__':
    unittest.main()
```

Och om vi kör den får vi:

```
.....
-----
Ran 5 tests in 0.001s
OK
```

Om vi har en bugg, som exempelvis att `eval_poly` alltid råkar returnera 0, blir resultatet:

```
.....FFFF
=====
FAIL: test_eval_poly_p0 (__main__.TestPoly)
-----

Traceback (most recent call last):
  File "test_polynomials.py", line 19, in test_eval_poly_p0
    self.assertEqual(eval_poly(p,0), 2, "Should be 2")
AssertionError: 0 != 2
```